DOCUMENTATION · REFERENCE

Features

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=FEATURES. This copy was made on 4 November 2026.

What QueryAPIGate does, feature by feature, with links to the reference for each. The README has the short version; the API reference has every detail.

Multi-database support

Tier 1 - PostgreSQL, MySQL, SQLite, DuckDB and ClickHouse: every feature, tested against a real server in CI, and covered by the compatibility promise. **Experimental** - H2, generic JDBC (Oracle, SQL Server, DB2, Snowflake and anything else with a driver jar) and MongoDB (`find()` queries): they work, with documented gaps. The same guard, pooling, parameter binding and output formats - JSON, NDJSON, XML, YAML, CSV, TSV, Excel and Parquet - apply to every one. What each supports - read-only enforcement, time limits, `allowed_tables`, schema browsing, streaming - is in the support matrix.

Files as a source - through DuckDB, Parquet, CSV and JSON files on local disk, S3, GCS, R2 or the web are queried where they are and published like any table, limited to the paths each connection lists (see publishing a bucket as an API).

Security and access control

QueryAPIGate is built to expose specific query results, not database credentials:

- API key authentication, with hashed key storage (SHA-256, never the raw secret) in `queryapigate.db`.
- Database connection passwords encrypted at rest (`QUERYAPIGATE_SECRET_KEY`), decrypted only in memory at the moment a connection is opened; a literal password on disk is never returned to a client either way.
- Scoped API keys: restrict a key to a set of connections, and/or to a specific allow-list of saved queries, independent of any connection grant - individual queries in that list can also be curated for write access, without granting it anywhere else reachable through the key.
- Workspaces (experimental, ADR 0023): several teams on one server, each with its own connections, saved queries, API keys, roles, destinations and exports - names unique within it - and its own history, audit entries, alerts and events. An administrator can have a role in one workspace only; a workspace's owner manages its members. Every route also answers at `/w/<workspace>/...`; a key works in its own.
- Example APIs: `queryapigate examples load` installs four worked scenarios - a reporting API, dashboard data, a streaming export and a partner integration - as collections, queries, roles and a ready-to-use API key per scenario, touching nothing of yours, and removable again with one command.
- Collections: file saved queries into named groups and grant a key a whole group - read-only, live, and every move that changes who can reach a query is audited with the keys that gained or lost access. Rename a collection without any key losing reach part-way; export one as a portable bundle and import it elsewhere.
- Named permission roles: a reusable template (connections, write access, queries, collections, rate limit, allowed IPs) copied onto a key once at creation, so a shared grant set for many keys lives in one place - editing or deleting a role afterward never affects a key already created from it.
- Read-only by default; write access (`INSERT` / `UPDATE` /DDL) is off unless explicitly enabled server-wide or granted per key, and can be narrowed further to specific write operations a key may perform.

- API key expiry (TTL) and revocation.
- Per-key rate limiting, on top of the server-wide, IP-based limit.
- IP allowlisting per key: pin a key to specific addresses or CIDR ranges.
- A row ceiling for streaming exports (`QUERYAPIGATE_STREAM_MAX_ROWS`), unbounded by default.
- A SQL guard that only allows single `SELECT` / `WITH` / `SHOW` / `DESCRIBE` / `EXPLAIN` statements through by default, with bound `:name` parameters rather than string-concatenated SQL.

```text
API Consumer
     |
     | API Key
     v
  QueryAPIGate
     |
     +-- Authentication
     +-- Authorization (connections + saved queries)
     +-- SQL guard
     +-- Rate limiting
     +-- Execution
     |
     v
  Database
```

This lets an organization hand an external client a key that can only ever reach one named query - never arbitrary SQL and never a whole connection - while internal callers keep broader, connection-level access.

MCP server for AI agents

The same saved queries are also tools an AI agent can call over MCP (the Model Context Protocol), served by a dedicated process (`queryapigate mcp`). It is the same governed core as REST, so an agent gets the same identity, grants, column masking, limits and audit as any app - enforced in the gateway, never asked of the model. It is how you let an agent query production data without handing it a database connection.

- **Saved queries as self-describing tools.** Each published query is an MCP tool carrying its parameters, their types, descriptions and an example, and the tables it reads - so an agent sees what each argument means, not just a name. A query can be hidden from MCP (`mcp: false`) while it keeps serving over REST.
- **Read-only by default; writes with approval.** The ad-hoc `execute_sql`, `list_tables` and `describe_table` tools are forced read-only regardless of the key's own write grant. Agent writes are held as requests and run only once a person approves them, with exactly the arguments shown (ADR 0008).
- **Masking-aware metadata.** `describe_table` and schema discovery honour column masking - a masked or dropped column is flagged, never shown as readable - so an agent can't even learn that hidden data exists.
- **Standard OAuth.** The MCP server is an OAuth 2.0 protected resource (RFC 9728): an agent signs in through your identity provider, and the token's audience is bound to the MCP server's own URL, so a token issued for it can't be replayed against REST or the database.
- **Every call recorded.** MCP runs land in the same run history, per-key metrics and tamper-evident audit log as every other caller.

The MCP server's own reference - tools, transport, configuration - is MCP server; to connect a client, see Let an AI agent call your saved queries via MCP.

Saved, versioned queries

A saved query bundles SQL, name, description, parameters, author, version, permissions and (optionally) cache configuration into one addressable resource: `GET /q/<name>`. Saving again under the same name creates a new version rather than overwriting the old one, so a change to the SQL doesn't silently change what's already live; `?version=1` still runs the prior one, and each run is recorded in the query's execution history.

Bound query parameters

```sql
SELECT * FROM film WHERE film_id = :id
```

```http
GET /q/film_by_id?id=7
```

Values are sent to the database separately from the SQL text, so they cannot be injected into it. Parameters can declare a type, default, required/optional, enum, numeric range, length and pattern - invalid input is rejected with a field-by-field `400` before it reaches the database.

Multiple response formats

JSON, NDJSON, XML, YAML, CSV, TSV, XLSX and Parquet are all available per request (`?format=`), so the same saved query serves application clients, reporting and data pipelines - a Parquet export keeps column types and is typically a quarter of the CSV's size.

Pagination and streaming

`?page=2&page_size=50` pages results with `X-Has-More` telling the caller whether more exist. For full exports, `?stream=true&format=csv` (or `tsv` / `ndjson`) streams the entire result straight from the database cursor rather than buffering it - verified end-to-end with 1,000,000-row results and flat server memory throughout (MySQL, PostgreSQL and ClickHouse); see Streaming exports for the measured numbers. `queryapigate export <query> --out '/exports/{name}_{date}.csv'` wraps the same streaming path as a CLI command for cron/systemd/Kubernetes CronJob to call directly - no server needs to be running; see Scheduled exports to a file.

Query caching

Saved queries can opt into HTTP-level caching: `cache_ttl`, `Cache-Control`, `ETag`, conditional requests and `304 Not Modified`, plus an `X-Cache: HIT` / `MISS` header. Never applied to a query that writes.

Rate limiting

A server-wide, IP-keyed limit (`QUERYAPIGATE_RATE_LIMIT`) and an independent, optional per-key limit can both be in effect at once - a request has to pass both. This lets different API consumers get different budgets:

```text
Internal dashboard  → generous server-wide limit, no per-key limit
Partner key          → 200/hour of its own
Trial key             → 20/hour of its own
```

Observability

Structured JSON logs (`QUERYAPIGATE_JSON_LOGS`) tagged with a request ID, per-query execution timing, slow-query warnings (`QUERYAPIGATE_SLOW_QUERY_THRESHOLD`), and Prometheus metrics at `/metrics` covering request/query counts and latencies, connection-pool occupancy and rate-limit rejections.

OpenAPI

`/openapi.json` is a complete, valid OpenAPI 3.0 document (checked in CI against the official validator) with every saved query listed as its own typed endpoint; `/docs` serves it through Swagger UI. A key's endpoint list is filtered to what that key can actually reach, so a query-scoped external key sees only its approved queries.

Administration UI

A built-in UI at `/console` covers the whole workflow: connection management, a SQL editor with syntax highlighting and schema browsing (click a table/column to insert it), query execution and preview, EXPLAIN, saved-query and version management, API key management, an audit log of administrative changes, per-query execution history, response inspection with a collapsible JSON tree for non-tabular results, a quick bar chart of any numeric result, and one-click "copy as curl" / "copy as TSV" for any result.

The UI has a collapsible sidebar (Data, Access, Observability; `Ctrl` + `B` toggles it) and a read-only **Settings** screen that shows every environment variable, its effective value and whether it was set or is the default (secrets are reported as configured or not, never shown), with a "Copy as .env" button. Theme, table density and the default result format are per-browser preferences.

```text
Connect Database
       |
       v
   Write SQL
       |
       v
  Test Query
       |
       v
  Save Query
       |
       v
Configure Access
       |
       v
  Expose API
```

Testing and quality

Unit tests, integration tests against real PostgreSQL, MySQL, ClickHouse, H2 and MongoDB servers (run in CI against service containers; the job fails if one skips), DuckDB integration tests that run unconditionally since it's embedded, property tests of the SQL guard with Hypothesis and fuzzing of the guards with Atheris and ClusterFuzzLite (fuzz/), CodeQL, static type checking with mypy, linting with ruff, and CI on every push - on Python 3.11 to 3.14, at the oldest dependency versions allowed, on a PostgreSQL store and under FIPS. See CONTRIBUTING.md.

What happens to every request

Every call - REST, MCP, events or an export - passes the same eight stages, as the README's second diagram follows one through:

1. **Identify the caller** - an API key, an admin token, a signed-in user's JWT or an OAuth token.
2. **Apply its limits** - rate limit, IP allow-list, expiry.
3. **Check the grant** - this query, this connection, read or write.
4. **Validate parameters** - types, ranges, lengths; values are bound, never spliced into the SQL.
5. **Guard the SQL** - one statement, read-only unless allowed, only the tables a key may see.
6. **Run it** - on a pooled connection, within the caller's time and row limits.
7. **Mask and format** - masking traced to the source columns, then the format asked for.
8. **Record it** - run history, metrics and live events; the audit log for changes.